

Dossier de projet professionnel pour le titre de concepteur développeur informatique



OWLBACK

Sommaire

1. Introduction.....	3
Présentation personnelle.....	3
Présentation du projet (in English).....	3
Compétences couvertes par le projet.....	3
2. Organisation et cahier des charges.....	
Analyse de l'existant	
Les utilisateurs du projet	
Les fonctionnalités attendues	
Contexte Technique	
Conception du projet.....	
Choix de développement	
Organisation du projet	
Architecture logicielle	
3. Conception du frontend de l'application.....	
Arborescence du projet	
Charte graphique	
4. Développement du frontend de l'application.....	
Arborescence	
Pages et composants	
Sécurités	
Exemple: Page de profil	
5. Conclusion.....	
En conclusion	
6. Annexes.....	

Introduction

Présentation personnelle

Bonjour, je m'appelle Mathéo Gass je suis passionné d'informatique depuis mes 5 ans, quand je suis arrivé au collège je me suis tout de suite pris de passion pour la programmation, et avec le temps j'ai voulu approfondir mes connaissances et pratiquant des langages de plus en plus bas niveau, jusqu'à toucher à l'assembleur et à développer de nombreux projets chez moi de tous types. Entrer à LaPlateforme m'a permis d'apprendre des nouveaux concepts et de varier mes connaissances.

Présentation du projet (in English)

Owlback is an expense management service available in two products: mobile application aimed at employees and Software as a Service (SAAS) platform for companies. It intends to make expense reporting easy to use and to automate the processing

of document information through AI in order to minimize errors, delays and time spent.

Compétences couvertes par le projet

Ce projet m'a permis de mettre en application une majorité des compétences que j'ai pu acquérir au cours de mes trois ans de formations. J'ai dans un premier temps établi un cahier des charges précédé par un brainstorming, nourri des discussions que j'ai pu avoir avec mon partenaire de projet ainsi qu'avec mes formateurs et mon entourage. Ensuite, j'ai travaillé sur la conception utilisateur à travers des maquettes graphiques et un user story, ce qui m'a amené à définir des entités, converties en tables dans une base de données relationnelle, appuyé par la méthode MERISE.

Une fois cela fait j'ai standardisé l'environnement de travail grâce à GitHub afin de renforcer notre organisation : gestion des tickets, versioning, et intégration continue via des scripts, pour ne citer que quelques-uns de ces outils. Docker est également présent afin de faire tourner les différents services autour du projet (ex : Redis) le plus efficacement possible.

En ce qui concerne le développement frontend, j'ai pu construire deux architectures différentes. Pour le développement mobile, j'ai privilégié React Native, orienté UI/UX pour simplifier l'expérience utilisateur. Pour le SAAS j'ai choisi React couplé à Inertia puisque la stack technique repose sur Laravel, framework sélectionné pour simplifier la charge

mentale ainsi que pour garder une continuité avec mon alternance.

Côté développement backend, comme mentionné précédemment, la stack choisie est Laravel. Pour gérer la plateforme web tel que le routing des pages comme le Dashboard ou la GED (Gestion Électronique des Documents), une partie API REST, sécurisée quant à elle par un système d'authentification par token (Laravel Sanctum). Des tests (notamment fonctionnels) ont également pu être préparés et exécutés avec PHPUnit afin de rendre le code le plus robuste possible.

Organisation et cahier des charges

Analyse de l'existant

Aujourd'hui il existe une pléthore de services traitant les notes de frais. C'est le cas de *Pennylane* ou encore de *Rydoo*. Elles offrent aussi, pour certaines, un service d'automatisation complète à l'IA ainsi qu'une comparaison avec les transactions bancaires. Cependant c'est en discutant avec mon entourage que je me suis rendu compte de plusieurs choses. Il y a non seulement une méconnaissance de ses applications mais également une récalcitance face à l'assimilation de ces dernières. L'entreprise Mesh Payments ressort confirme selon leur propres études ces propos.



Owlback peut être une première étape à l'intronisation de ces applications, se voulant simple d'utilisation auprès des utilisateurs mais aussi auprès des entreprises car chaque fonctionnalité a pour but d'être accessible et intégrée rapidement à l'usage du quotidien.

De plus, selon nos recherches, d'après l'URSSAF, *20% des notes de frais comportent une erreur* (justificatifs manquants, doublons, dépenses non conformes), entraînant des retards de remboursement ou des refus purs et simples. Pour les salariés, cela signifie avancer des sommes parfois conséquentes sur leur capital personnel. Selon l'entreprise APOGEA, c'est aussi le risque d'un redressement URSSAF ou d'un litige prud'homal pour l'employeur.

Les utilisateurs du projet

Ce projet possède deux parties distinctes : une application mobile et une plateforme web type SAAS.

L'application mobile sera utilisée par un type d'utilisateur : les salariés de l'entreprise une fois qu'ils sont invités. En effet, un compte utilisateur est généré par la direction de l'entreprise. L'espace auquel accèdent les employés, permet le dépôt des notes de frais, la complétion d'informations si nécessaire ou la gestion des fichiers.

En parallèle de la page vitrine pour faire découvrir ce service, l'interface web est destinée aux entreprises ayant souscrit à un abonnement. Elle permet d'avoir un suivi temporel sur plusieurs critères, une GED ou bien l'intégration d'un nouveau collaborateur au travers d'une invitation générée par mail.

Les fonctionnalités attendues

Application Mobile :

- Ecran de démarrage : Lorsque l'utilisateur lance l'application, il est dirigé vers cette page, comportant un bouton de redirection. S'il est déjà connecté, l'utilisateur est envoyé vers le tableau de bord. Dans le cas contraire, il est dirigé vers la page de connexion.
- Ecran de connexion : L'utilisateur doit renseigner son email et son mot de passe pour être redirigé vers le tableau de bord. À la connexion, le backend génère un token pour le collaborateur, afin qu'il puisse exécuter les requêtes API.

- Ecran de tableau de bord : Cette page permet de récupérer un ensemble d'informations relatives à l'utilisateur telles que les notifications, le total des dépenses du mois en cours, les trois derniers documents, les activités, ainsi qu'un graphique permettant d'analyser la répartition des types de frais.
- Ecran de profil : Sur cette page, l'utilisateur peut consulter et modifier ses informations, supprimer son compte, se déconnecter, ou encore personnaliser certains paramètres de l'application.
- Ecran de GED : Cette page permet de visualiser les documents importés par l'utilisateur ou L'IA (via le canal de l'email). L'utilisateur peut aussi retrouver un document par le biais d'une barre de recherche.
- Nav bar : Une barre de navigation globale permettant à l'utilisateur d'accéder rapidement aux pages les plus importantes.
- Ecran de dépense : Permet au collaborateur de suivre ses dépenses de manière chronologique et selon plusieurs critères.
- Ecran de détail d'un document : Cette page offre la possibilité à l'employé d'accéder à un document importé afin de le consulter ou de l'éditer.
- Scan : L'utilisateur peut prendre en photo ou importer un document afin qu'il soit traité par l'application.

Interface Web :

- Page d'accueil : Il s'agit de la page vitrine du site web.

- Page d'inscription : Permet à une entreprise de s'inscrire grâce à un formulaire.
- Page de connexion : Permet à une entreprise de se connecter grâce à un formulaire.
- Page de tableau de bord : Si l'entreprise a souscrit à un abonnement, cette page lui permet d'accéder à diverses statistiques basées sur différents critères, comme la chronologie ou le type de note de frais, ainsi qu'à ses documents.
- Page de GED : Cette page est accessible uniquement si l'entreprise a souscrit à un abonnement. Elle permet de retrouver tous les documents importés par les collaborateurs, ainsi que les informations qui y sont associées. De plus, il est possible de télécharger un fichier individuellement, ou un dossier dans son intégralité.
- Page de Documentation : Documentation technique liée à l'utilisation de l'API accessible uniquement aux abonnés.
- Page de profil : Permet de consulter ou modifier les informations de l'entreprise, de créer et inviter un utilisateur (à condition d'être abonné), de souscrire à un abonnement, ou encore de modifier ou supprimer un utilisateur.
- Page récapitulative : Une fois par mois, une page récapitulative est accessible, permettant d'agréger l'ensemble des dépenses afin de les consulter, les

éditer et surtout de remonter d'éventuelles erreurs de traitement. A sa validation un fichier CSV est généré, puis envoyé à l'adresse email du comptable.

Contexte technique

L'application mobile doit être accessible principalement sur IOS et Android mais pour le projet nous nous sommes focalisés sur IOS.

Le SAAS doit être accessible sur les navigateurs modernes, tels que Chrome, Firefox etc...

Conception du projet

Choix de développement

 Application Mobile : Pour l'application mobile, le langage utilisé est Typescript, associé au framework React Native couplé à Expo.

Ce choix technique répond à plusieurs objectifs. Typescript apporte un typage statique, ce qui permet de détecter un

grand nombre d'erreurs dès la phase de développement plutôt qu'à l'exécution. Cela facilite la maintenance du code, améliore la lisibilité et rend le travail en équipe plus fiable grâce à l'auto-complétion. Il répond aussi à certains comforts techniques personnels pour la phase de développement



React Native a été retenu car il permet de développer une application mobile cross-platform (IOS et Android) à partir d'une base de code unique, ce qui représente un gain de temps considérable. L'ajout d'Expo simplifie encore davantage le processus : il me permet notamment de pouvoir simuler mon application directement sur mon téléphone sans avoir à prendre du temps sur des émulateurs (XCode ou Android studio).



Interface Web et backend : Le langage utilisé côté frontend est également Typescript, afin d'affiner mes connaissances, et toujours pour les mêmes raisons mentionnées précédemment.



Le framework choisi est React couplé à Inertia, bibliothèque permettant de contrôler une mono-page (SPA) depuis le serveur sans la mise en place d'une API REST dédiée pour la partie web. React est aussi un choix personnel pour sa

prise en main rapide et surtout pour le recyclage de code déjà existant.



Côté backend, le langage retenu est PHP, parfaitement adapté au framework Laravel, qui a été sélectionné pour la conception de l'interface web et de l'API.

En effet, Laravel a été sélectionné d'une part pour garder une continuité avec mon alternance, afin de toujours monter en compétence, mais aussi pour son environnement technique vaste, stable et productif, permettant d'accélérer au mieux le développement. C'est notamment le cas de l'ORM Eloquent, qui simplifie les interactions avec la base de données, ou encore de la structure claire du framework, où les responsabilités sont nettement décomposées, que ce soit pour l'interface web ou pour la consommation de l'API REST par l'application mobile. Cela fait de ce framework un outil exceptionnellement solide et intéressant.

Par ailleurs, dans le cadre de ce projet j'ai fait usage de plusieurs outils externes à savoir :

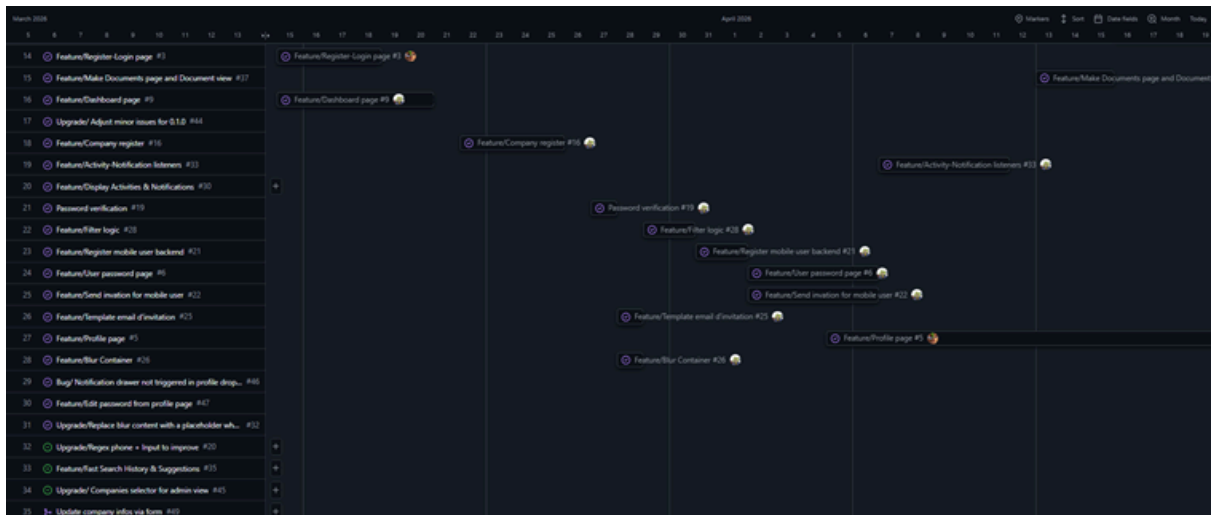
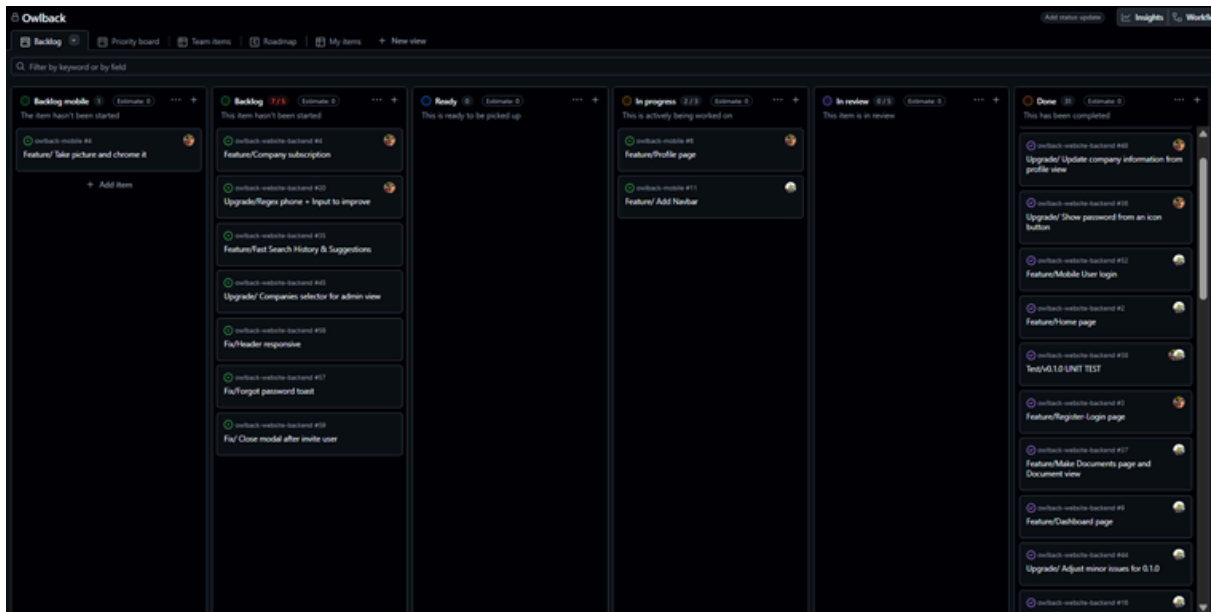
- **Visual studio code** pour écrire mon code
- **Postman** pour tester mes API
- **Github** et son environnement pour stocker, versionner, et organiser le projet
- **NPM** pour installer les paquets coté frontend
- **Composer** pour aussi installer les paquets coté backend

- **Figma** pour maquetter l'application mobile
- **Draw.io** pour schématiser la base de données
- **Expo GO** pour émuler mon projet
- **Docker** pour conteneuriser l'ensemble du projet (à l'exception du React Native)
- **Gemini AI et Claude code** pour la génération du logo, des images fictive du site vitrine et le développement

Organisation du projet

Afin d'assurer un suivi régulier de l'avancement du projet, une réunion d'environ 30 minutes est organisée une fois par semaine, en présentiel ou en visioconférence, permettant de faire le point sur les blocages rencontrés, les avancées réalisées et de définir la suite du développement.

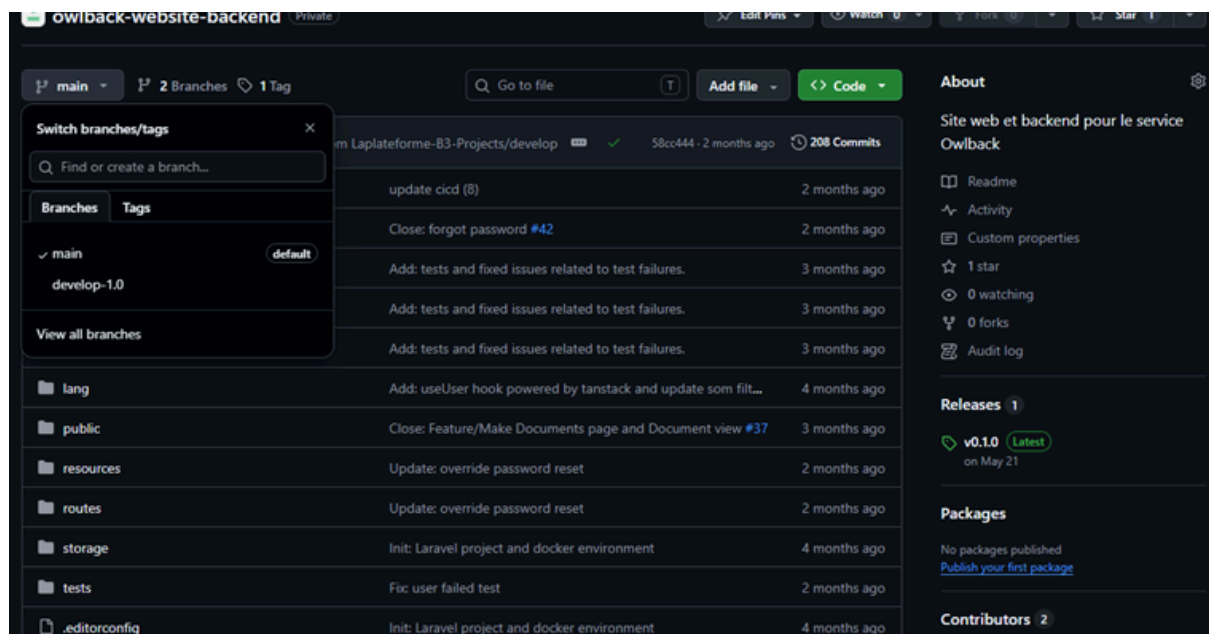
La gestion du code source s'appuie sur GitHub, avec deux dépôts distincts réunis au sein d'une organisation GitHub : l'un dédié à l'interface web et au backend, l'autre à l'application mobile. Le suivi des tâches est assuré via GitHub Projects, sur lequel les tickets sont organisés selon une nomenclature par type (« Feature/ », « Upgrade/ », « Bug/ », « Hotfix/ », « Test/ »), ainsi que par version du projet, et répartis selon leur état d'avancement. Un diagramme de Gantt, également intégré à GitHub Projects (Roadmap), permet d'assurer un suivi chronologique des différentes fonctionnalités.



De plus, en ce qui concerne les branches, chaque nouvelle fonctionnalité est développée sur une branche dédiée. Le projet repose sur deux branches principales : une branche *main*, garantissant une version stable du projet, et une branche *develop*, destinée à centraliser les fusions des nouvelles fonctionnalités avant leur passage en production.

Enfin, les commits suivent également une nomenclature définie, permettant d'identifier rapidement la nature de chaque modification apportée au projet : "Add:" pour l'ajout d'une

nouvelle fonctionnalité ou d'un élément, "Update:" pour la modification ou l'amélioration d'un élément existant, et "Close:" pour la clôture d'un ticket ou la résolution d'un problème. Cette convention facilite la lecture de l'historique du projet et la traçabilité des changements effectués.



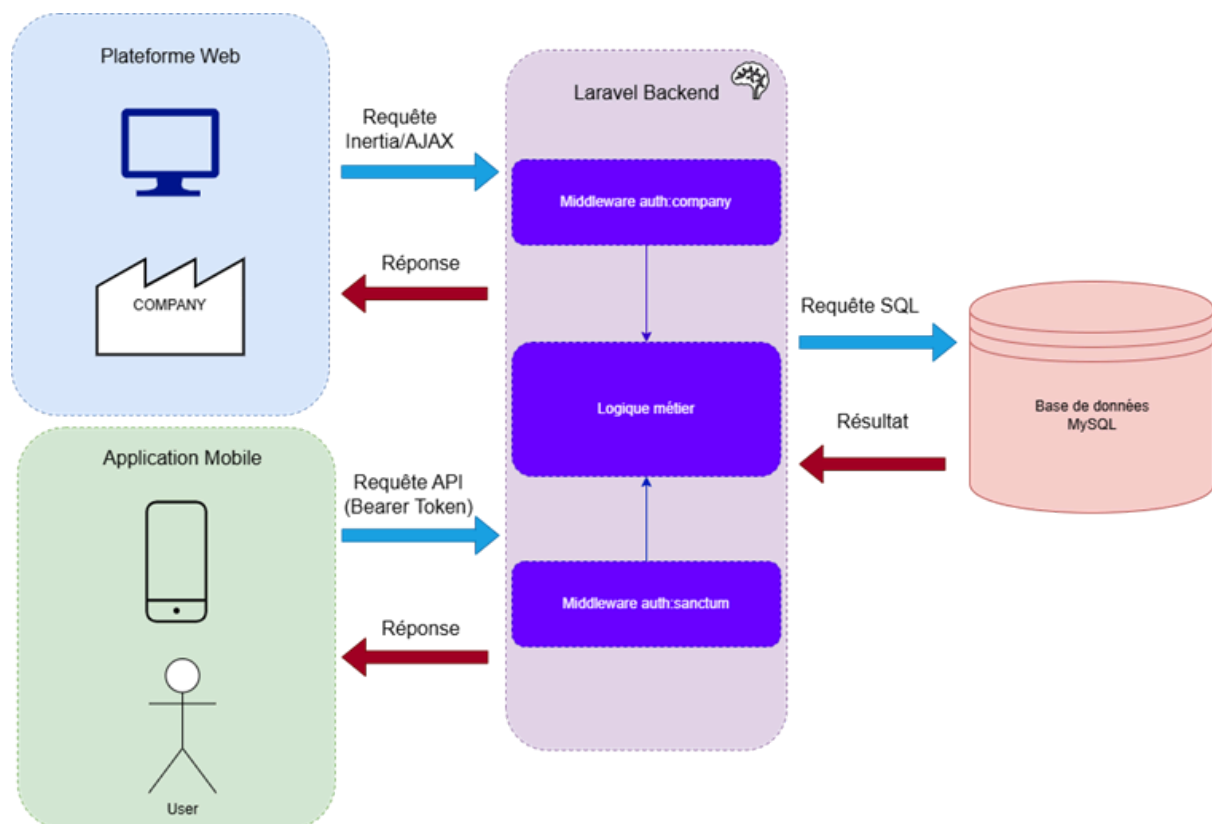
Architecture logicielle

Pour chaque partie du projet, l'acteur à l'origine des appels diffère selon le canal utilisé.

Côté Web, les appels au backend sont liés aux comptes Entreprises : l'authentification repose sur le mécanisme de session de Laravel, où un cookie de session (accompagné d'un token CSRF assurant la protection contre les attaques cross-site) permet au serveur d'identifier l'utilisateur (soit une société) connecté à chaque requête Inertia, évitant un token applicatif. L'entreprise est alors immédiatement retrouvée côté serveur, typiquement par le biais du middleware « *auth:company* ». Seule exception à cette règle : la création du

mot de passe d'un salarié, action qui, par nature, précède l'existence d'une identité utilisateur individuelle authentifiable, et ne peut donc pas reposer sur ce mécanisme.

Côté mobile en revanche, l'absence de navigateur (et donc de gestion automatique des cookies et du CSRF) impose une authentification stateless : chaque appel est associé à un utilisateur authentifié individuellement, via un Bearer Token porté explicitement dans chaque requête API.



Conception des frontends du projet

Arborescence du projet

Frontend Web :

Cette partie est monolithique, c'est-à-dire que l'ensemble du code est partagé et regroupé au même endroit. Voici un aperçu de l'arborescence frontend :

L'arborescence est organisée en différent dossier. Chacun de ces dossiers et ces contenus favorisent la scalabilité en limitant au mieux les responsabilités.

Le dossier *components* tous les composants exploités par les pages du site. Qu'ils soient atomiques (boutons, inputs personnalisés) ou plus complexes (cartes, modales). Le dossier *data* concentre les informations statiques propres au site, ainsi que l'ensemble des schémas de validation des formulaires. Les *hooks* personnalisés sont regroupés dans un même emplacement. Ils encapsulent majoritairement la logique d'interaction avec le backend, ce qui évite la redondance possible des appels.

Quant à *utils*, il contient toutes les fonctions utilitaires génériques. Enfin, le dossier *types* centralise les typages statiques utilisés par les composants, garantissant une cohérence des structures de données à travers l'application.

Application Mobile :

Ici l'arborescence se résume en un seul bloc de projet puisqu'il ne contient que la partie mobile. A l'instar du web, cette partie possède des dossiers similaires comme *components*, *hook* et *utils*. Cependant, c'est le dossier *app* qui officie comme routeur de l'application, via l'Expo Router : contrairement au Web où le routage est défini côté backend, ici l'arborescence des fichiers à l'intérieur de *app* détermine directement les écrans et les routes disponibles, sans configuration de routeur manuelle.

Charte graphique

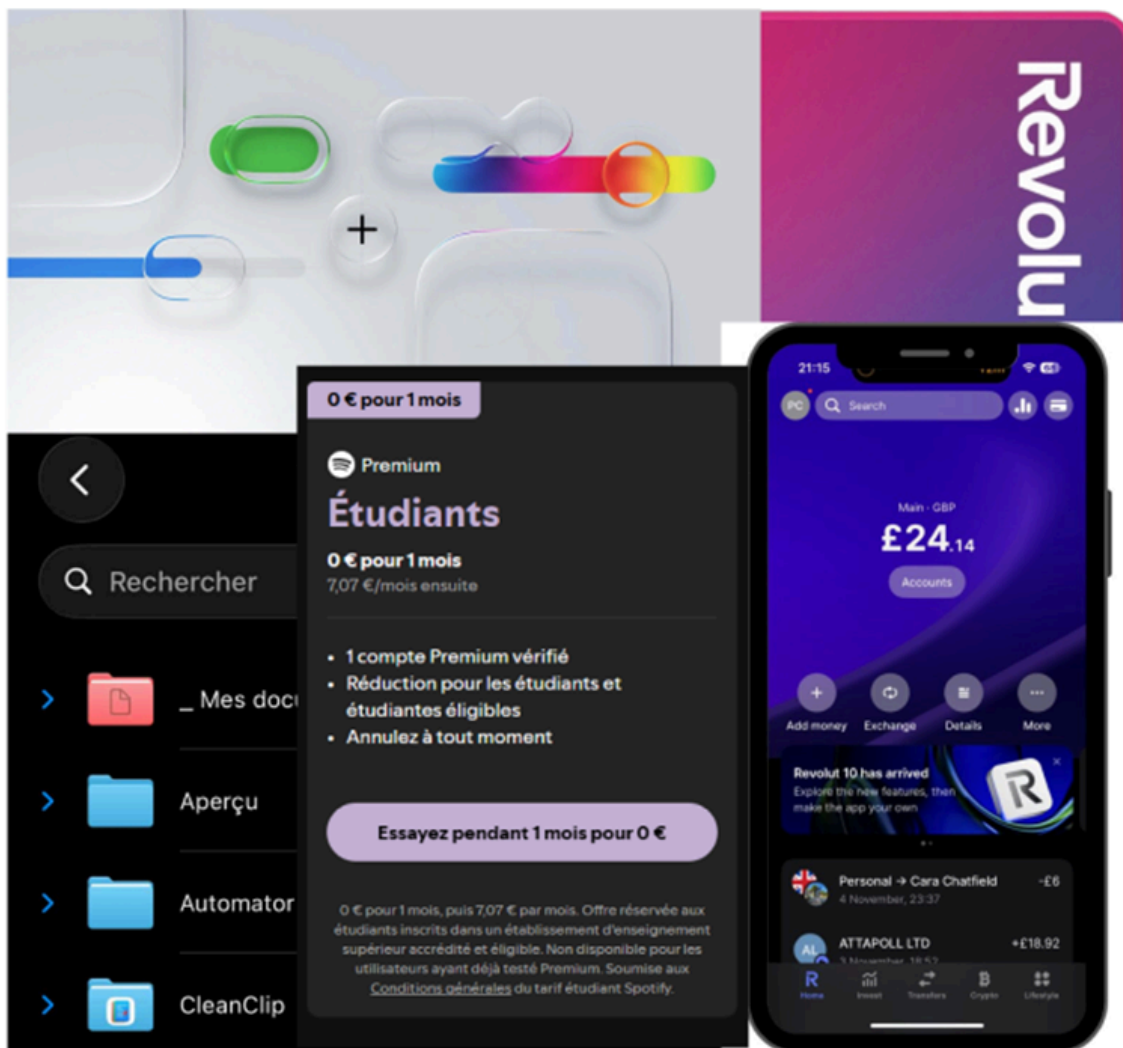
Dans un premier temps, j'ai dû réaliser le logo représentant Owlback. Les principales idées directrices étaient qu'il devait inspirer la confiance et la sagesse avec une identité visuelle forte. Je l'ai donc fait générer par Gemini, ce qui m'a permis d'itérer rapidement sur plusieurs propositions visuelles sans compétences préalables en design graphique. L'envie de mettre en avant une mascotte m'avait aussi paru évidente puisque j'essaye d'en avoir une pour chacun de mes projets et qu'il s'agissait d'une conclusion logique après diverses réflexions avec mon partenaire de projet.



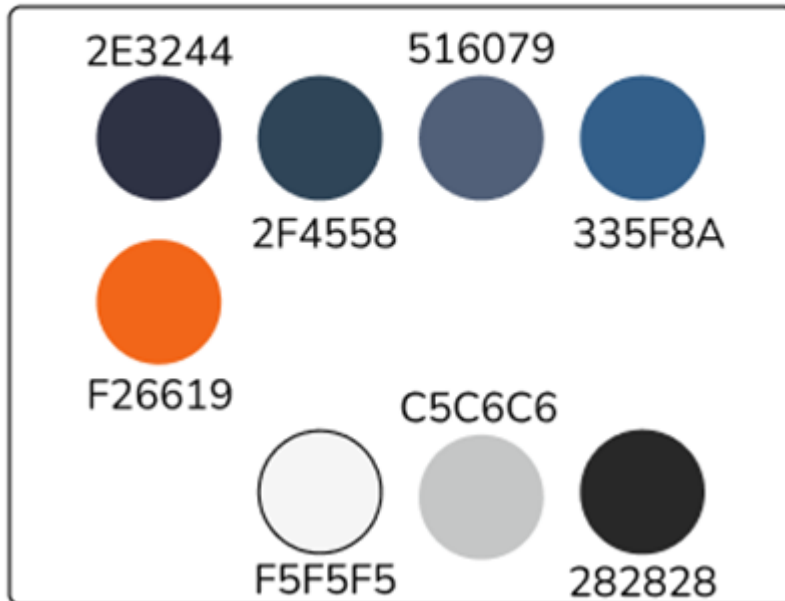
L'application Owlback trouve son inspiration au travers de divers éléments déjà existant. J'ai donc établi un *moodboard* de mes inspirations (Revolut, Apple liquid glass, Spotify etc..) avant d'étoffer les couleurs définies en amont. L'objectif était d'avoir un rendu cohérent, élégant et surtout professionnel. Concernant la typographie, la police *Roboto* est utilisée pour les titres afin d'apporter une forte lisibilité et un caractère moderne, tandis que *Verdana* ou *Nunito* sont envisagées pour

le corps de texte, privilégiant le confort de lecture.

Moodboard



Palette de couleur



Font de l'application mobile Roboto

Verdana ou Nunito

Maquette

La construction de la maquette s'est faite sur l'outil *Figma*. Le wireframe a permis de poser les bases en associant praticité et confort d'utilisation, notamment à travers l'agencement des éléments. La maquette finale a permis d'harmoniser les tons de couleurs, en affirmant les choix du *moodboard* pour un rendu aussi élégant que possible.

[Voir annexe page : XX-XX]

Conception backend du projet

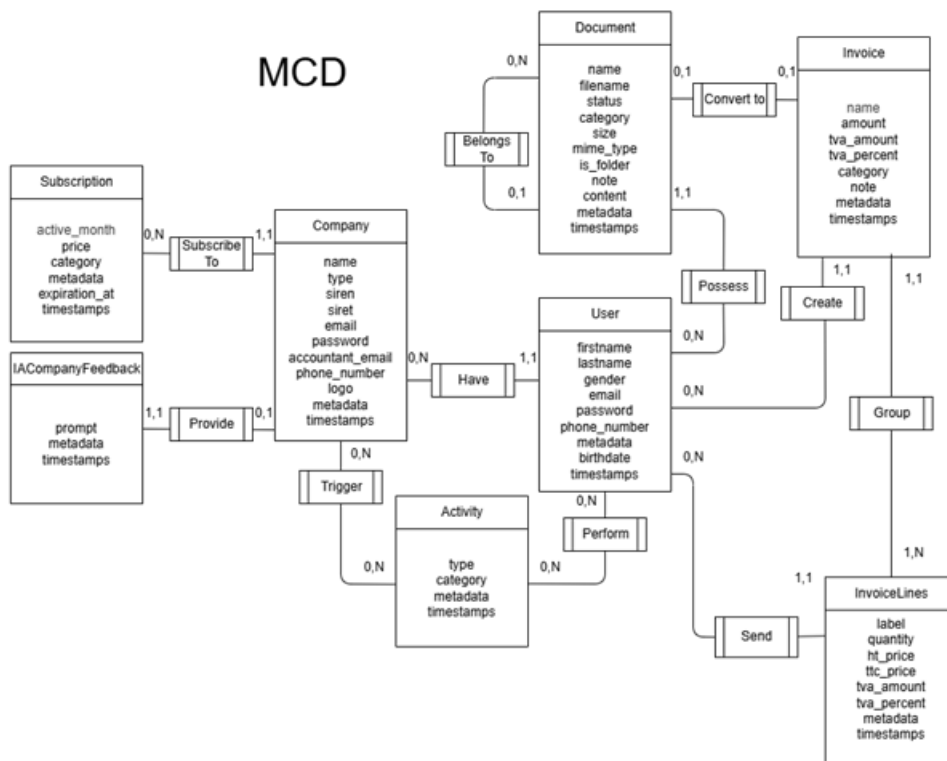
La base de données

Pour le projet, il est impératif qu'une base de données soit présente et bien structurer afin d'avoir une compréhension solide des enjeux du service Owlback.

Ayant beaucoup travaillé alternance avec les bases de données relationnelles, que ce soit en formation ou en alternance, le choix s'est naturellement porté vers MySQL. De plus Eloquent est un ORM (Object Relational Mapping) Laravel conçu pour manipuler des données relationnelles. Partir sur du NoSQL serait un non-sens de partir en se privant d'un des avantages du framework.

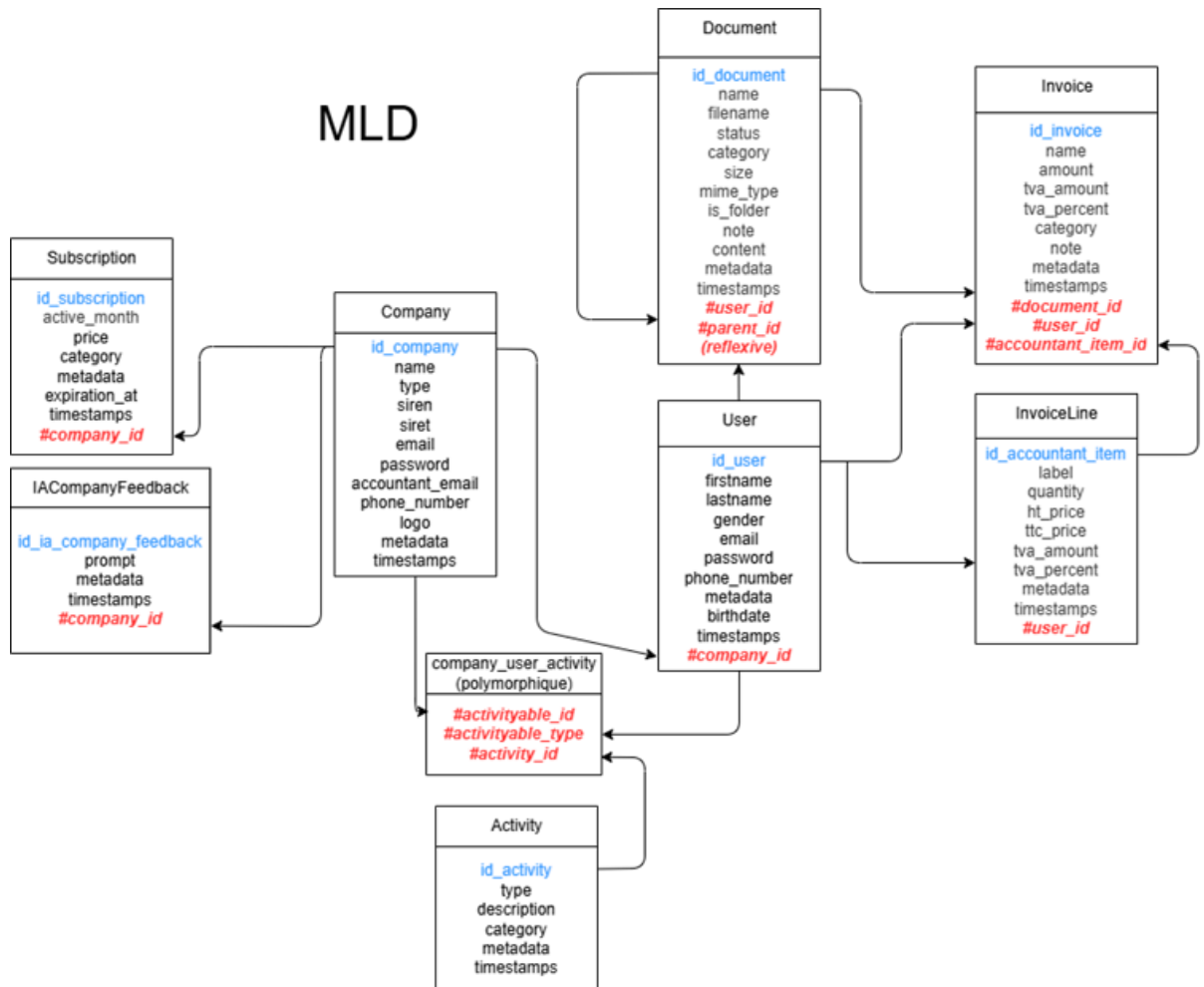
Pour concevoir ma base de données j'ai utilisé la méthode Merise. Elle se divise en trois étapes successives :

- Modèle conceptuel des Données (MCD) : Ce schéma m'a permis d'identifier clairement les entités principales, à savoir *User*, *Company*, *Document*, *Invoice*, *InvoiceLine*, *Activity*, ou encore *Subscription*, ainsi que les relations entre elles par l'intermédiaire de verbes associatifs et de cardinalités.



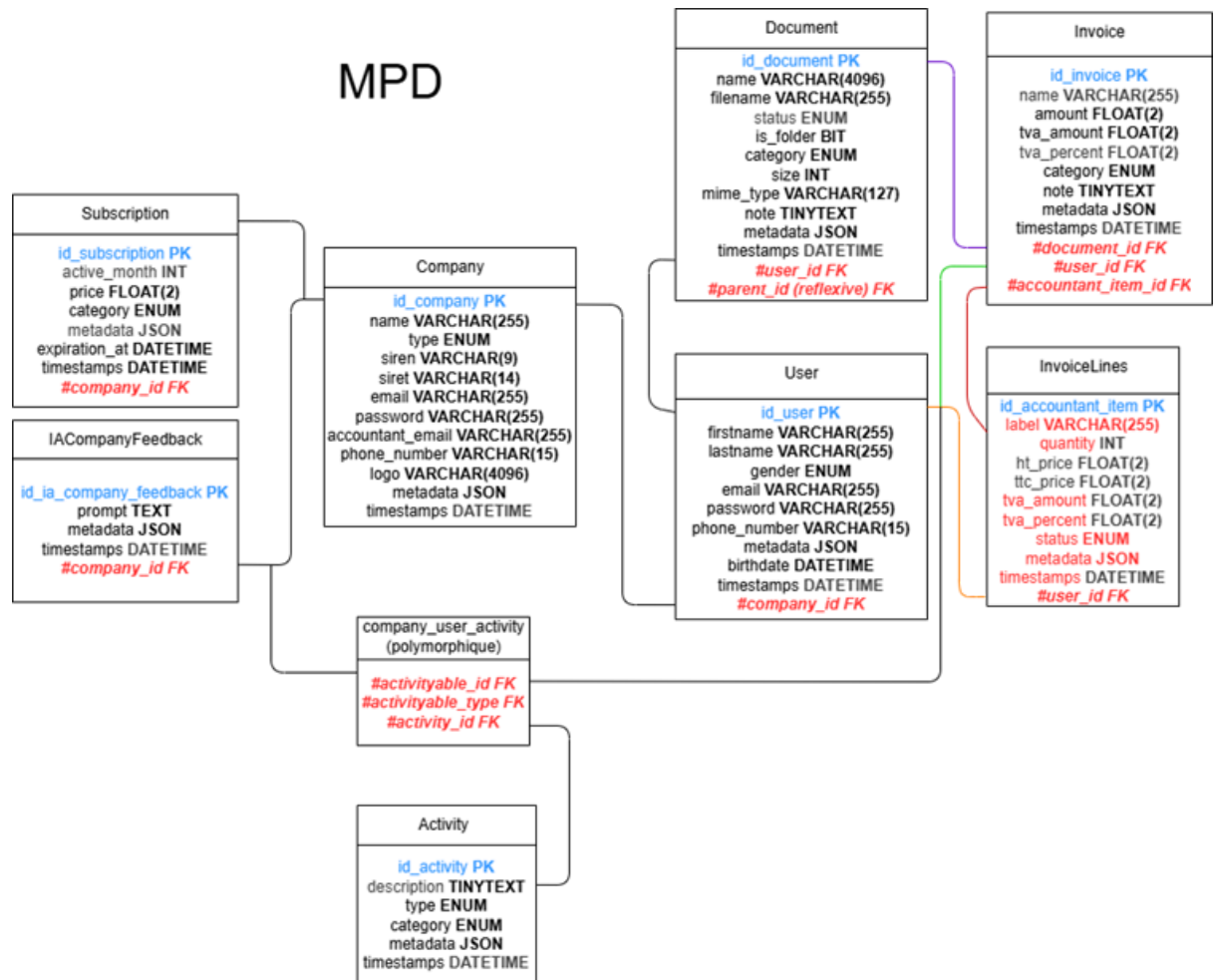
- Modèle Logique des Données (MLD) : ce schéma renforce les associations prédéfinies en ajoutant les clés primaires et étrangères. Les tables pivots sont mises en avant lorsqu'une relation N–N est présente ; dans le cas de ce projet, aucune table de liaison classique n'a été nécessaire. En revanche, une relation polymorphique (un modèle pouvant être associé à plusieurs autres modèles différents à travers une seule association) a été établie entre les entités *User*, *Company* et *Activity*, afin qu'une même activité puisse être rattachée indifféremment à un utilisateur ou à une société sans dupliquer la structure. Pour l'illustrer, j'ai représenté une table dédiée à cette association.

MLD



- Modèle Physique des Données (MPD) : ce schéma résulte directement du modèle logique. Les champs des tables sont désormais typés (VARCHAR, ENUM, FLOAT, DATETIME...), avec leurs contraintes précises,

et il servira de base directe aux fichiers de migration
Laravel.



Une fois que tout a été conceptualisé, des fichiers de migration Laravel ont été créés pour chacune des tables identifiées. Une migration est un fichier de code décrivant la structure d'une table. Elle permet de créer ou de faire évoluer le schéma de la base de données sans intervention manuelle, tout en garantissant que ce schéma puisse être répliqué à l'identique sur n'importe quel environnement.

A screenshot of a code editor window with a dark background and light-colored text. The code is a Laravel migration for creating a 'users' table. It uses the 'Schema::create' method with a function parameter that takes a 'Blueprint' object. The function defines various table columns and their constraints: 'id' (primary key), 'firstname' and 'lastname' (strings), 'gender' (enum with values from 'GenderEnum::values()'), 'email' (string, unique), 'phone_number' (string, 25 characters, nullable), 'password' (string, nullable), 'password_created_at' (timestamp, nullable), 'birthdate' (date, nullable), 'metadata' (json, nullable), 'rememberToken' (boolean), and 'timestamps' (datetime with on/off timestamps). The code is numbered from 1 to 14 on the left side of the editor.

```
1 Schema::create('users', function (Blueprint $table) {  
2     $table->id();  
3     $table->string('firstname');  
4     $table->string('lastname');  
5     $table->enum('gender', GenderEnum::values());  
6     $table->string('email')->unique();  
7     $table->string('phone_number', 25)->nullable();  
8     $table->string('password')->nullable();  
9     $table->timestamp('password_created_at')->nullable();  
10    $table->date('birthdate')->nullable();  
11    $table->json('metadata')->nullable();  
12    $table->rememberToken();  
13    $table->timestamps();  
14 }
```

Développement du backend de l'application

Organisation

Le backend a été conceptualisé dans l'objectif d'être exploité à la fois par la plateforme Web et par l'application mobile. Pour ce faire, deux fichiers de routage distincts ont été mis en place afin de séparer clairement l'appartenance des routes et l'affiliation des *Controllers* associés.

En effet, lorsqu'un même modèle est sollicité par les deux plateformes, les *Controllers* sont nommés différemment afin de garder une compréhension et une exécution du code claires. Par exemple, le modèle Document, exploité à la fois côté Web pour diverses actions, et côté mobile pour la même finalité, ne transite pas par le même Controller : côté Web, les routes passent par *DocumentController*, tandis que côté mobile, elles passent par *DocumentApiController*. Cette distinction se justifie par des formats de réponse différents (pages Inertia pour le Web, JSON pour l'API mobile), même si la logique métier sous-jacente reste comparable.

Les méthodes peuvent être sensiblement les mêmes mais afin d'éviter la duplication inutile du code, elles s'articulent toujours selon le principe de *Action Pattern*:

- Validation des données via des classes *Request* dédiées
- Une ou plusieurs *Actions* portant le code métier
- Renvoi des données ou non selon la pertinence de la requête

Arborescence

Il a déjà été mentionné que Laravel est une application monolithique. Cependant, le backend est architecturé en différentes couches, chacune apportant un module de code exploitable et réutilisable.

L'application se compose de plusieurs dossiers et sous-dossiers, mais cette section abordera exclusivement les plus importants, à savoir :

- Actions : contient les logiques métier de l'application, organisées par modèle.
- Routes : agrège l'ensemble des routes consommées par l'application, réparties entre les fichiers Web et API évoqués précédemment.
- Controllers : contient l'ensemble des controllers, chargés d'orchestrer le bon déroulement du code
- Middleware : contient l'ensemble des middlewares personnalisés, permettant de vérifier une condition avant d'autoriser l'accès à la suite du traitement.
- Models : contient l'ensemble des modèles Eloquent de la base de données, avec leurs méthodes propres et les relations qui les lient entre eux.
- Tests : contient l'ensemble des tests unitaires (vérifiant une unité de code isolée, comme une Action) et fonctionnels (vérifiant un comportement de bout en bout, comme une requête HTTP complète) effectués à ce jour.

Fonctionnement des requêtes Web/API

Il existe deux acteurs pour Owlback. L'un représente des sociétés dites *Company* et l'autres des utilisateurs dites *User*.

Les *Companies* naviguent à travers le site web via des requêtes HTTP classiques. Celles-ci sont envoyées, associées à la bonne route, puis validées par les middlewares présents, avant d'être transmises aux *Controllers* qui traitent la requête. Les données renvoyées sont, la plupart du temps, des réponses HTML, JSON ou des redirections. Pour les *Users*, le processus est identique, à l'exception près qu'un *Bearer Token* est requis afin de vérifier l'authenticité de l'utilisateur. Les réponses envoyées sont alors systématiquement de nature *JSON*.

Les différents statuts apparants dans ce projet sont :

- 200 : *OK* Indique que la requête a réussi
- 201 : *CREATED* Indique que la requête a réussi et une ressource a été créé
- 204 : *NO – CONTENT* Indique que la requête a bien été effectué et qu'il n'y a pas de contenu à envoyer
- 400 : *BAD REQUEST* indique que le serveur n'a pas pu comprendre la requête à cause d'une syntaxe invalide
- 401 : *UNAUTHORIZED* bien qu'il indique *non-autorisé*, la sémantique de cette réponse correspond à

non-authentifié. Le client doit s'authentifier afin d'obtenir la réponse demandée

- 403 : *FORBIDDEN* indique que le client n'a pas les droits d'accès au contenu, donc le serveur refuse de donner la véritable réponse
- 404 : *NOT FOUND* Indique que le serveur n'a pas trouvé la ressource demandée.
- 422 : *UNPROCESSABLE ENTITY* indique que la requête est syntaxiquement correcte, mais que les données envoyées ont échoué à la validation
- 429 : *TOO MANY REQUESTS* : indique que l'utilisateur a envoyé trop de requêtes dans un laps de temps donné, et

Développement du frontend de l'application

Arborescence

L'application est organisée selon l'architecture MVC (Model-View-Controller) de Laravel afin de séparer les responsabilités entre la logique métier, les données et l'interface utilisateur. Le code est structuré en plusieurs dossiers dédiés aux contrôleurs, modèles, migrations, composants React Native, routes et services.

L'interface est découpée en plusieurs écrans accessibles via une navbar. Chaque écran correspond à une fonctionnalité précise, ce qui facilite la maintenance et l'évolution de l'application.

Pages et composants

L'application est composée de plusieurs pages principales :

Écran de démarrage: page sur laquelle chaque nouvel utilisateur arrive avant.

Authentification: inscription et connexion des utilisateurs.

Page de profil: consultation et modification des informations personnelles.

Page de paramètres: modifications des paramètres utilisateurs, et des paramètres liés à l'application.

L'interface est développée avec React Native en utilisant des composants réutilisables (boutons, champs de saisie, cartes, listes, modales, etc.). Cette approche permet de limiter la duplication du code et de garantir une interface cohérente et plus facile à développer.

La communication entre React Native et Laravel est réalisée via une API REST sécurisée.

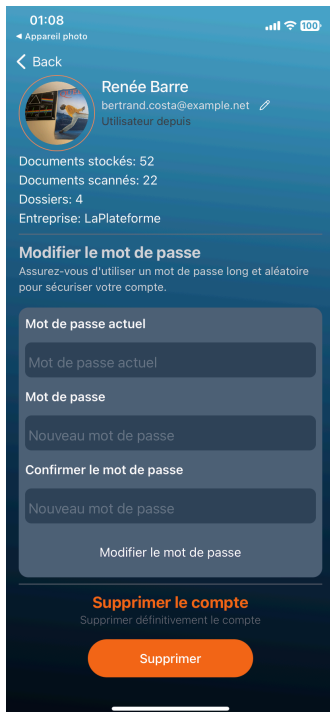
Sécurité

Plusieurs mécanismes ont été mis en place afin de sécuriser l'application :

- Authentification des utilisateurs
- Contrôle des autorisations avant l'accès aux ressources
- Validation des données côté serveur avec Laravel
- Protection contre les requêtes malveillantes grâce aux fonctionnalités natives de Laravel
- Stockage sécurisé des mots de passe par hachage
- Utilisation de jetons d'authentification pour sécuriser les échanges entre le client React Native et l'API.

Ces mesures garantissent la confidentialité des données et limitent les risques d'accès non autorisés.

Exemple: Page de profil



Voici un exemple de la page de profil, page par laquelle vous pouvez modifier votre adresse mail, modifier votre photo de profil, et modifier votre mot de passe. Vous pouvez également supprimer votre compte depuis celle-ci, cependant, une confirmation vous sera demandée avant.

Conclusion

Annexes